

CNIT 372 Final Presentation - Complete Script

Nursing Home Management Database System

1. Project Title & Team Roles

Our team consists of four members:

Weitong Ding – Database schema design, stored procedures, triggers, constraints, and exception handling.

Qingyang Mei – ERD creation, entity relationships, and sample SQL queries.

Nathaniel Ruminski – ERD creation, triggers, constraints,

Member 4 – Testing, debugging, documentation, and video editing.

2. Problem Context & Significance

- Population aging leads to increased demand for elderly care.
- Many nursing homes still rely on **paper records or fragmented systems**.
- Problems include:
 - Misplaced records
 - Delayed communication
 - Medication errors
 - Poor coordination among staff
 - Families unable to access timely updates

2. Problem Context & Significance

Our Solution — CareConnect

- A centralized digital system integrating:
 - Resident records
 - Room assignments
 - Staff information
 - Medication + Care plans
 - Visitor management

-

Impact:

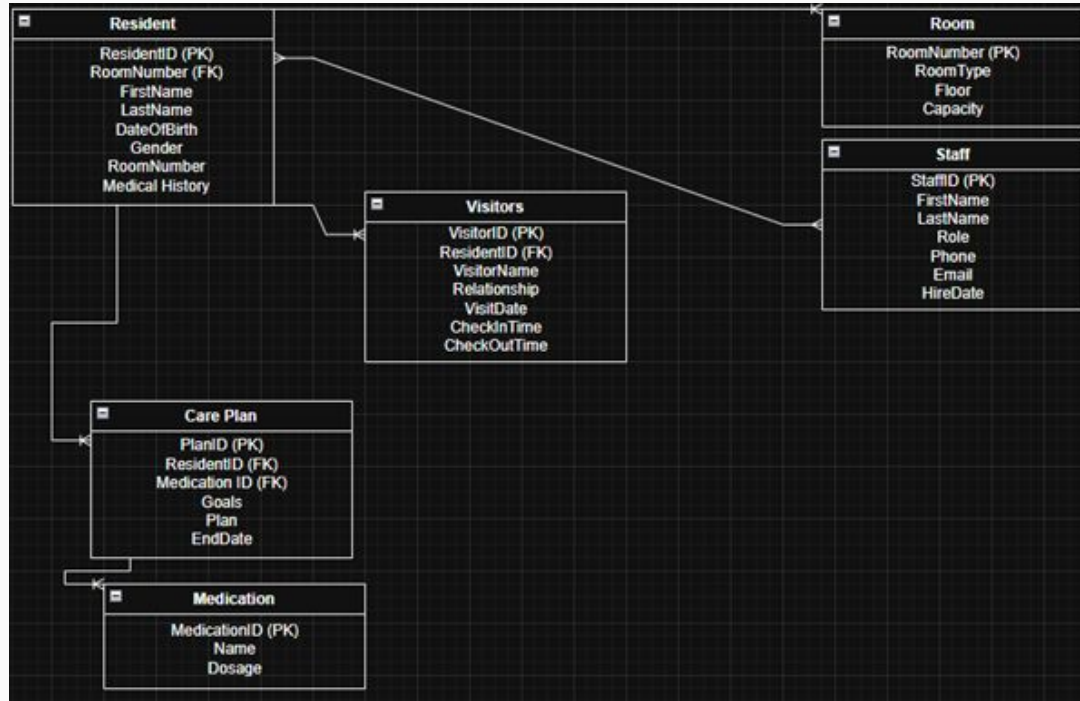
- Improves efficiency
- Reduces medical/administrative errors
- Enhances communication and trust between staff, residents, and families

3. Data Source & Database Complexity

Complexity Features:

- Fully normalized schema
- 6 core tables: Room, Staff, Resident, Visitors, Medication, CarePlan
- Many-to-many relationships (Residents ↔ Medications via CarePlan, Residents ↔ Visitors)
- Hierarchical structure: floors & room layout
- Each table includes **50+ rows** (total dataset > 300 records)
- Supports real nursing home operations (care, scheduling, visits)

4. ERD (Entity Relationship Diagram)



5.Database Schema Walkthrough

Resident Table

Field	Notes
ResidentID (PK)	Unique resident ID
RoomNumber (FK)	Links to Room
FirstName / LastName	Personal info
DateOfBirth	Used for age calculation
Gender	'Male'/'Female'
MedicalHistory	TEXT medical notes

5.Database Schema Walkthrough

Room Table

Field	Notes
RoomNumber (PK)	Unique room ID
RoomType	Single / Double
Floor	Floor number
Capacity	1-2 persons

5.Database Schema Walkthrough

Staff Table

Field	Notes
StaffID (PK)	Unique staff ID
Role	Doctor / Nurse / Caregiver / Therapist
Contact Info	Phone + Email
HireDate	Staff onboarding date

6. 10 Questions Summary

Question 1: Residents per room type. This allows us to see what residents are in which rooms.

Question 2: Which residents have >3 visitors? This question allows us to see which residents will have less than 3 visitors. This allows us to know if this nursing home should expect a constant flow of guests or someone every once and a while.

Question 3: Who are the 5 oldest residents? This allows our staff to know the 5 oldest residents and to keep in mind that they might need a bit more care than others.

Question 4: What is the gender distribution? This allows the employees to know the gender distribution to help sort where residents live. This is good to keep a good mix of people.

Question 5: What is the room occupancy vs capacity? This question helps staff answer questions to new people trying to get a nursing home room for their loved one.

Question 6: How many visitors per day? This lets the staff know how many visitors each person has per day.

Question 7: Care plan per resident. Staff need to know each residents care plan to help keep them as comfortable as possible.

Question 8: Which room has the most residents? This is to keep track of who is living with just their wife/husband or with their kids as well inside their room.

Question 9: Add routine note to male residents. Here we did a test to just add a routine note to male residents to keep track of them better.

Question 10: Keyword search in medical history. Keywords searching is important to allow a staff member to quickly look something up.

7. triggers, constraints, exception handling

Triggers

1. Trigger: `trg_resident_validate`

Ensures a resident is at least 18 years old before the row is saved.

Uses `MONTHS_BETWEEN` to calculate age.

Prevents invalid records from being inserted or updated.

2. Trigger: `trg_resident_audit`

Every time the MedicalHistory field changes, the system automatically inserts a row into `Resident_Audit`

Creates a full change history for medical information.

7. triggers, constraints, exception handling

Constraints

Constraint `chk_gender`

Ensures the *Gender* column contains only valid values ("Male" or "Female").

Prevents bad data from entering the system.

7. triggers, constraints, exception handling

Exception handling

1. Duplicate Resident (INSERT)

```
WHEN DUP_VAL_ON_INDEX THEN  
  
    DBMS_OUTPUT.PUT_LINE('Resident  
already exists.');
```

Prevents inserting same ResidentID twice.

2. Non-existent Resident (UPDATE / DELETE)

Before updates or deletes:

```
IF NOT resident_exists(p_id) THEN  
  
    RAISE_APPLICATION_ERROR(...)  
  
END IF;
```

Avoids operations on invalid or missing records.

7. triggers, constraints, exception handling

Exception handling

3. Data Type Errors

```
WHEN VALUE_ERROR THEN  
  
    DBMS_OUTPUT.PUT_LINE('Invalid input  
data type.');
```

Catches invalid input and informs the user.

Exception handling

4. Foreign Key Constraint Handling during DELETE

```
WHEN OTHERS THEN  
  
    IF SQLCODE = -2292 THEN  
  
        DBMS_OUTPUT.PUT_LINE('Delete blocked:  
related rows exist.');
```

Prevents deleting a resident with dependent records (e.g., visitors, care plans).

7. triggers, constraints, exception handling

Exception handling

5. Triggers and Procedures Using Application Errors

Several places use:

```
RAISE_APPLICATION_ERROR(-200xx, 'message');
```

Produces readable, custom error messages.

Helps students and users understand what went wrong.

8. Ethical/security considerations

Protection of Sensitive Medical Data

The system stores medical history, personal information, and room assignments, which are highly sensitive.

Ethical responsibility: protect resident privacy and prevent unauthorized access.

Security requirement: use database constraints, triggers, and controlled procedures to ensure only valid, accurate updates occur.

Auditability and Accountability

Every change to medical history is recorded with:
User ID, Timestamp, Before and after values

Helps detect misuse or suspicious activity.

Supports ethical obligations for transparency and oversight.

9. Live demo



10. Lessons learned, challenges, and team reflections

Writing Maintainable PL/SQL

Stronger Understanding of Error Handling

Integrating Code from Multiple Team Members

Managing Data Dependencies

10. Lessons learned, challenges, and team reflections

Realistic Experience With Database Development

Growth in Technical and Soft Skills